

AI 3603 ARTIFICIAL INTELLIGENCE: PRINCIPLES AND TECHNIQUES

By: Kaiyang Xu (523030910085)

HW#: 2

November 15, 2025

I. TASK 1: REINFORCEMENT LEARNING IN CLIFF-WALKING ENVIRONMENT

A. Sarsa

On-policy TD(0) with target using the next action a' sampled from current policy:

$$\delta_t = r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t), \quad (1)$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \delta_t. \quad (2)$$

If done = True, we do not bootstrap ($Q(\text{terminal}, \cdot) = 0$), hence $\delta_t = r_t - Q(s_t, a_t)$.

The **Sarsa algorithm** works as follow:

1. Initialize $Q(s, a) = 0$; set $\alpha, \gamma, \epsilon, \epsilon_{\min}, \lambda$.
2. For each episode:
 - (a) Reset s ; select a using epsilon-greedy (take $\arg\max_a Q(s, a)$ with probability $1 - \epsilon$, otherwise pick a random action uniformly).
 - (b) Loop until termination: execute a to obtain (s', r, done) ;
 - If done: $\delta = r - Q(s, a)$; update $Q(s, a) \leftarrow Q(s, a) + \alpha \delta$; terminate.
 - Otherwise: select a' using epsilon-greedy on $Q(s', \cdot)$ (same rule as above), compute $\delta = r + \gamma Q(s', a') - Q(s, a)$ and update; set $s \leftarrow s', a \leftarrow a'$.
 - (c) Decay $\epsilon \leftarrow \max(\epsilon_{\min}, \lambda \epsilon)$.

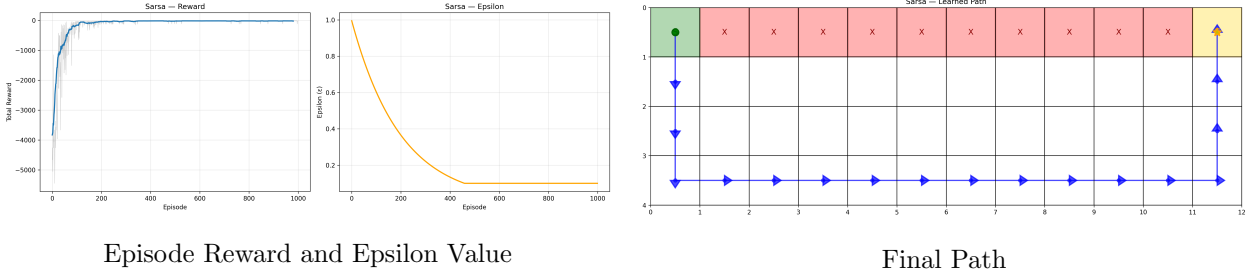


FIG. 1: Result of Sarsa algorithm

B. Q-Learning

Off-policy TD(0) with greedy target:

$$\delta_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t), \quad (3)$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \delta_t. \quad (4)$$

If done = True, then $\delta_t = r_t - Q(s_t, a_t)$.

The **Q-Learnin algorithm** works as follow:

1. Initialize Q and hyperparameters.
2. For each episode: reset s and loop until termination:
 - (a) Choose a using epsilon-greedy (take $\arg\max_a Q(s, a)$ with probability $1 - \epsilon$, otherwise random) and interact to obtain (s', r, done) .

- (b) Compute $\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ (if done, the target is r).
- (c) Update $Q(s, a)$, set $s \leftarrow s'$, and decay ϵ .

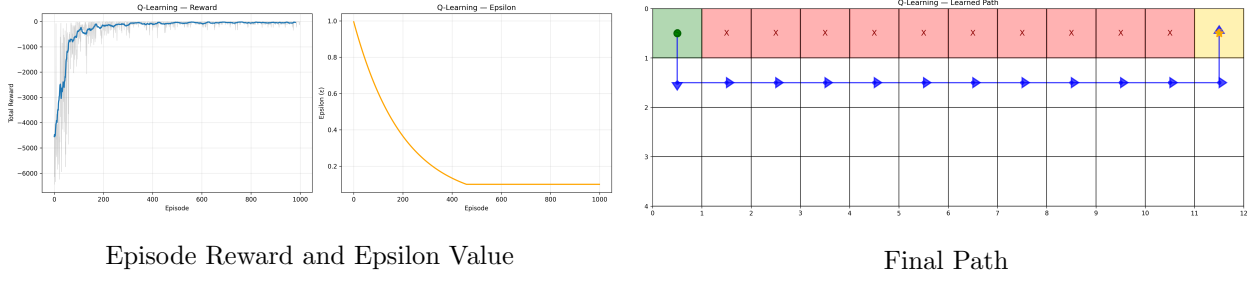


FIG. 2: Result of Q-Learning algorithm

C. Dyna-Q

After each real interaction, perform two types of updates:

1. **Real update** (same as Q-Learning): $\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ (if done, the target is r).
2. **Model learning and planning**: record a model $\mathcal{M}(s, a) = (\hat{s}, \hat{r}, \hat{d})$; repeat n_{plan} times, sample a previously seen $(\tilde{s}, \tilde{a})$, query $(\tilde{s}', \tilde{r}, \tilde{d}) = \mathcal{M}(\tilde{s}, \tilde{a})$ and perform a “virtual” Q-learning update (terminal samples also do not bootstrap).

The **Dyna-Q algorithm** works as follow:

1. Initialize Q and an empty model $\mathcal{M}$; set n_{plan} .
2. For each episode: run real interactions and update Q ; then perform n_{plan} model-based updates.

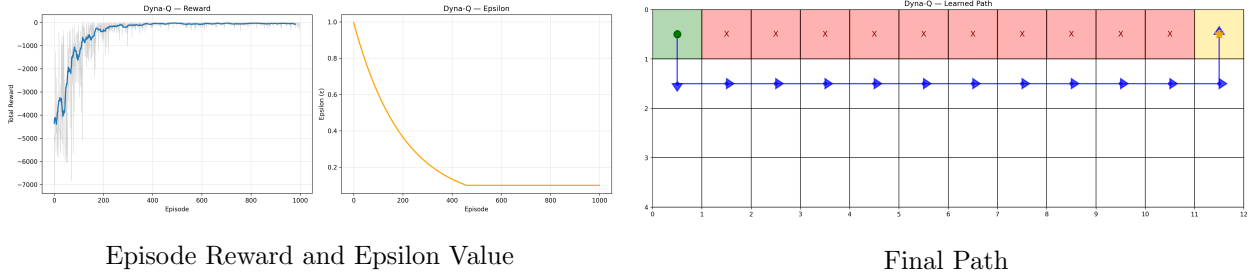


FIG. 3: Result of Dyna-Q algorithm

D. Result Analysis

- Environment: **CliffWalking-v0** (4x12 grid).
- Training: each agent is trained for $N = 1000$ episodes; maximum steps per episode: 500. All three algorithms use the same hyperparameters for a fair comparison:

1. `SarsaAgent(action_space, alpha=0.1, gamma=0.99, epsilon=1.0, epsilon_min=0.1, epsilon_decay=0.995)`
2. `QLearningAgent(A, alpha=0.1, gamma=0.99, epsilon=1.0, epsilon_min=0.1, epsilon_decay=0.995)`
3. `Dyna_QAgent(A, alpha=0.1, gamma=0.99, epsilon=1.0, epsilon_min=0.1, epsilon_decay=0.995, n_planning=20)`

- Seed: fixed random seed (`RANDOM_SEED = 0`) for reproducibility in examples.
- Metrics recorded per episode: cumulative episode reward, current ϵ .
- Evaluation: after training, set $\epsilon = 0$ to roll out a greedy path for visualization.

The comparison of rewards and epsilons through **Sarsa**, **Q-Learning**, **Dyna-Q** is shown in FIG. 4.

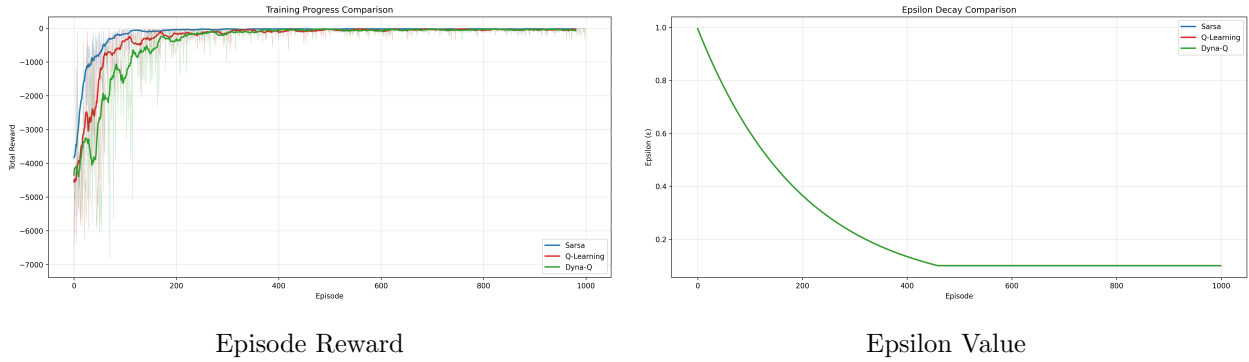


FIG. 4: Comparison of Rewards and Epsilons

In conclusion, we can see

- **Path difference (Sarsa vs Q-Learning):** In most runs, **Sarsa learns a more conservative path** that stays farther from the cliff, while **Q-Learning tends to hug the cliff** and find the shortest (optimal) path.
 - *Reason:* Sarsa is on-policy and its target uses the next action value under the current ϵ -greedy policy, $r + \gamma Q(s', a')$. With nonzero ϵ , a' carries the risk of exploratory moves that may fall into the cliff; this risk is reflected in the target and pushes the policy to prefer safer routes.
 - By contrast, Q-Learning is off-policy and bootstraps with a greedy target $r + \gamma \max_{a'} Q(s', a')$, ignoring the exploration risk in the next step. This biases learning toward the maximal-value (shortest) path along the cliff edge.
- **Model-based (Dyna-Q) vs model-free (Sarsa/Q-Learning) efficiency:** **Dyna-Q typically learns faster** (steeper reward improvements and earlier convergence) than Sarsa or Q-Learning given the same interaction budget.
 - *Reason:* After each real step, Dyna-Q performs n_{plan} model-based updates, effectively multiplying value propagation and improving sample efficiency. In the deterministic CliffWalking environment, the learned one-step model is accurate, so these planning updates are especially effective.
 - With identical hyperparameters and per-episode ϵ decay, the speedup observed in reward curves primarily stems from planning rather than differences in exploration schedules.

II. DEEP REINFORCEMENT LEARNING

A. DQN

Standard DQN approximates the optimal action-value function $Q^*(s, a)$ with a parameterized network $Q_\theta(s, a)$. It uses off-policy bootstrapping and experience replay to break correlations between sequential data and stabilize updates via a separate target network Q_{θ^-} . For a batch of transitions $(s_i, a_i, r_i, s'_i, d_i)$ the temporal-difference target is:

$$y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\theta^-}(s'_i, a'). \quad (5)$$

The loss minimized is mean squared error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B (y_i - Q_\theta(s_i, a_i))^2. \quad (6)$$

After every gradient step the online network parameters θ are updated; periodically (every `target_network_frequency` steps) the target parameters are set $\theta^- \leftarrow \theta$.

The Q-network defined in `QNetwork` is a Multi-Layer Perceptron (MLP):

- Input dimension: 8 (flattened state vector).
- Hidden layers: 120 units (ReLU), then 84 units (ReLU).
- Output layer: 4 linear units (one per discrete action) producing $Q(s, \cdot)$.

This lightweight architecture is sufficient for LunarLander; deeper networks could be explored for performance gains or stability.

Exploration uses ε -greedy action selection. ε decays linearly from `start_e` to `end_e` over a fraction `exploration_fraction` of total timesteps:

$$\varepsilon_t = \max \left(\varepsilon_{\text{start}} + (\varepsilon_{\text{end}} - \varepsilon_{\text{start}}) \frac{t}{T_{\text{decay}}}, \varepsilon_{\text{end}} \right). \quad (7)$$

This maintains steady reduction of exploration rather than the rapid early drop typical of exponential decay.

A circular replay buffer of size `buffer_size` stores transitions $(s, a, r, s', d, info)$ allowing uniform random sampling of batches. Off-policy sampling decorrelates data and allows reuse of past experience increasing sample efficiency.

The training loop proceeds for `total_timesteps` interactions. Prior to `learning_starts` only data collection occurs. Afterward, every `train_frequency` environment steps a mini-batch of size `batch_size` is sampled and one gradient update is performed.

The **DQN algorithm** works as follow:

1. Initialize replay buffer $\mathcal{D}$.
2. Initialize online network parameters Q_θ .
3. Initialize target network parameters $Q_{\theta^-} \leftarrow Q_\theta$.
4. Set global step counter $t = 0$.
5. Reset environment to obtain initial state s .
6. **Repeat until** $t \geq \text{total_timesteps}$:
 - (a) Compute current exploration rate ε_t via the linear schedule.
 - (b) Select action a :

- With probability ε_t : sample a random action.
 - Otherwise: $a = \arg \max_{a'} Q_{\theta}(s, a')$.
- (c) Execute action a ; observe next state s' , reward r , done flag d , and info.
- (d) Store transition (s, a, r, s', d) in $\mathcal{D}$.
- (e) If $d = 1$ (episode ended): reset environment to obtain new s' .
- (f) Set $s \leftarrow s'$ and increment global step $t \leftarrow t + 1$.
- (g) If $t > \text{learning_starts}$ and $t \bmod \text{train_frequency} = 0$:
- i. Sample mini-batch $B = \{(s_i, a_i, r_i, s'_i, d_i)\}$ uniformly from $\mathcal{D}$.
 - ii. Compute TD targets: $y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\theta^-}(s'_i, a')$ (no gradient).
 - iii. Compute loss: $\mathcal{L} = \frac{1}{|B|} \sum_i (y_i - Q_{\theta}(s_i, a_i))^2$.
 - iv. Perform one gradient descent step on $\mathcal{L}$ updating θ .
 - v. If $t \bmod \text{target_network_frequency} = 0$: set $\theta^- \leftarrow \theta$.

B. Result Analysis

We use the hyperparameter shown in TABLE I. We don't tune the structure of the Q network or utilize multiple continuous frames of the game.

TABLE I: Hyperparameter Configuration

Parameter	Value	Description
Learning rate (α)	1.0×10^{-3}	Step size for Adam optimizer updates
Replay buffer size	10,000	Maximum capacity of experience memory
Discount factor (γ)	0.99	Importance weighting of future rewards
Target network frequency	500 steps	Synchronization interval for target Q-network
Batch size	128	Number of experiences sampled per gradient step
Initial epsilon (ϵ_0)	1.0	Starting probability for random action selection
Final epsilon ($\epsilon_{\min}$)	0.1	Lower bound for exploration rate
Exploration fraction	0.5	Proportion of training dedicated to exploration phase
Learning starts	10,000 steps	Initial period for experience collection only
Train frequency	5 steps	Interval between network optimization steps
Total timesteps	500,000	Overall training budget in environment steps

FIG 5 shows the training progress over 500,000 timesteps. The curve demonstrates successful learning from random initialization (approximately -500) to stable positive performance (approximately 0).



FIG. 5: Episode Return

We evaluate our Q network on 5 episodes, and achieve:

- **Average episode return:** -143.22
- **Individual episode return:**
 - Episode 1: -33.43
 - Episode 2: -195.74
 - Episode 3: -175.68
 - Episode 4: -211.14
 - Episode 5: -100.11
- **Best episode:** -33.43
- **Success rate:** 40% (2 of 5 test episodes resulted in successful landings)

C. Some Improvement

We use the hyperparameter shown in TABLE I. This time, we utilize two continuous frames of the game instead of one frame each time.

FIG 6 shows the training progress over 500,000 timesteps. The curve demonstrates successful learning from random initialization (approximately -200) to stable positive performance (approximately $+200$).

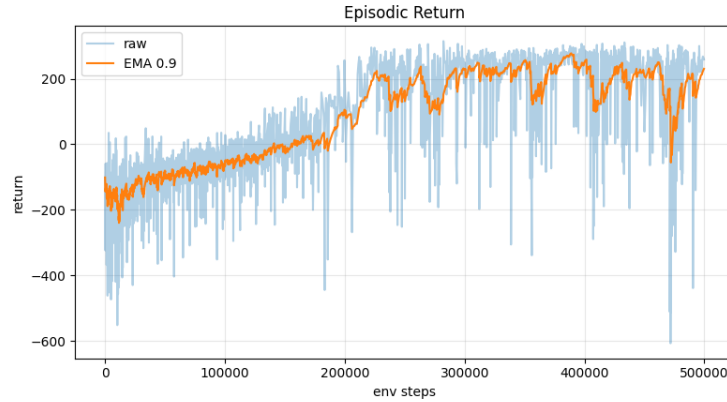


FIG. 6: Episode Return

We evaluate our Q network on 5 episodes, and achieve:

- **Average episode return:** 249.78
- **Individual episode return:**
 - Episode 1: 277.06
 - Episode 2: 235.38
 - Episode 3: 239.36
 - Episode 4: 228.79
 - Episode 5: 268.30
- **Best episode:** 277.06

- **Success rate:** 100% (all test episodes resulted in successful landings)

In conclusion, utilizing continuous frames of the game instead of one frame each time provides a fundamental advantage in partially observable environments, as it equips the Q-network with the necessary temporal context to learn a robust and effective policy.

III. IMPROVE EXPLORATION SCHEMA

UCB balances exploration and exploitation by adding an optimism bonus to the value estimate. A common form is

$$\text{UCB}(s, a) = \hat{Q}_\theta(s, a) + c\sqrt{\frac{\ln N(s)}{1+N(s, a)}},$$

with $c>0$ controlling exploration and $N(s), N(s, a)$ approximate visit counts. During data collection choose

$$a_t = \arg \max_a \text{UCB}(s_t, a),$$

while keeping the DQN learning target unchanged. In continuous state spaces, replace counts by uncertainty proxies, e.g., a small ensemble of Q-heads and use the predictive standard deviation σ_a :

$$\text{UCB}(s, a) = \hat{Q}_\theta(s, a) + c\sigma_a.$$

Brief pros/cons:

- Pros: targeted exploration, often more sample-efficient than ε -greedy; principled optimism.
- Cons: relies on uncertainty quality; c needs tuning; counts/ensembles add computation and memory.